AWS Data-Source Governance (S3/RDS) — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — AWS Data-Source Governance (S3/RDS): Microsoft Purview 5

What you get . . . 5

What this delivers . . . 5

The deployment model . . . 5

Outcome . . . 6

1. Solution Overview — AWS Data-Source Governance (S3/RDS): Microsoft Purview 6

What this solution is . . . 6

Why this matters . . . 6

The two AWS sources this covers . . . 6

The Data Map, collections, and roles . . . 7

Credentials and network — safe by design . . . 7

Licensing and the evolving portal . . . 7

What this is not . . . 7

The deployment model . . . 8

2. Requirements Guide — AWS Data-Source Governance (S3/RDS): Microsoft Purview 8

2.1 A Microsoft Purview account (Data Map) provisioned on an Azure subscription . . . 8

2.2 Amazon S3 / Amazon RDS connector availability confirmed for your tenant . . . 8

2.3 Least-privilege data-governance roles on the Data Map collection . . . 9

2.4 An AWS account inventory of the S3 buckets and RDS instances in scope, with owners . . . 9

2.5 Amazon S3 trust path — a cross-account IAM role (ARN + external ID), read-only . . . 9

2.6 Amazon RDS credential via Azure Key Vault — never in code . . . 10

2.7 Network path to each source (integration runtime + AWS network/firewall) . . . 10

2.8 A classification approach (system + any custom classifications) . . . 10

2.9 A pilot scope and change control . . . 10

Checklist . . . 11

Global Requirements — Microsoft Purview Solutions 11

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 12

5. Compliance, records, privacy & legal stakeholders . . . 12

6. A pilot scope and change control . . . 13

3. Implementation Guide — AWS Data-Source Governance (S3/RDS): Microsoft Purview 13

Step 1 — Assess current state (read-only) . . . 13

Step 2 — Design the collection and role model . . . 14

Step 3 — Establish the S3 trust path (cross-account IAM role, safe by design) . . . 14
Step 4 — Establish the RDS credential path (Key Vault, safe by design) . . . 14
Step 5 — Record the pilot scope . . . 15
Step 6 — Register the pilot AWS sources into the Data Map . . . 15
Step 7 — Configure and pilot the scan (bounded scope, pilot-first) . . . 15
Step 8 — Review scanned assets and classifications . . . 15
Step 9 — Assign ownership on the AWS assets . . . 15
Step 10 — Validate against the pilot gate . . . 15
Step 11 — Expand across the AWS estate (enforce) . . . 16
5. Testing Guide — AWS Data-Source Governance (S3/RDS): Microsoft Purview 16
Test 1 — Account and roles are ready . . . 16
Test 2 — Amazon S3 / RDS connectors are available . . . 16
Test 3 — The S3 trust path works (role assume + Test connection) . . . 17
Test 4 — The RDS credential path works (Key Vault + network + Test connection) . . . 17
Test 5 — The scan succeeds and returns assets . . . 17
Test 6 — AWS assets are classified . . . 17
Test 7 — Scanned AWS assets have owners . . . 17
Test 8 — Consolidated readiness gate . . . 17
Pass/fail gate . . . 17
6. Troubleshooting Guide — AWS Data-Source Governance (S3/RDS): Microsoft Purview 18
A. Cannot reach the account / token fails . . . 18
B. Amazon S3 / Amazon RDS connector isn't offered . . . 18
C. S3 source registration or Test connection fails . . . 19
D. RDS source registration or Test connection fails . . . 19
E. Scan runs but classifies nothing . . . 20
F. AWS request/egress cost higher than expected . . . 20
G. Classifications look wrong . . . 20
H. Ownership / gate not passing . . . 20
I. Diagnostics quick reference . . . 21
J. When to escalate . . . 21
7. Best Practices — AWS Data-Source Governance (S3/RDS): Microsoft Purview 21
8. Deliverables Summary — AWS Data-Source Governance (S3/RDS): Microsoft Purview 22
Documentation . . . 22
Named artifacts . . . 23
PowerShell scripts . . . 23
What the buyer receives . . . 24

Outcome 24
Appendix A — PowerShell Scripts 25

Executive Summary — AWS Data-Source Governance (S3/RDS): Microsoft Purview

Most data-governance efforts stop at Microsoft 365, and the AWS estate — customer data in S3 buckets, application databases on Amazon RDS — sits outside the map, unclassified and ungoverned. You cannot protect, retain, or grant access to data you cannot see. This solution closes that blind spot: it **registers, scans, and classifies your AWS S3 and RDS sources into the Microsoft Purview Data Map**, so AWS data becomes a mapped, classified, owned part of the same governed estate as the rest of your data.

This is a complete, do-it-today package written for an engineer bringing AWS into Purview for the first time. It delivers the AWS connector workflow end-to-end — the safe **role-based trust** for S3, a **Key Vault** credential for RDS, a bounded pilot scan, classification review, and ownership — with every artifact named once and reused everywhere, and every scan proven on a bounded pilot before the AWS estate is touched. Because the Purview multicloud connectors are evolving and consumption-billed, the guide cites the method and flags every capability "verify your licensing/tenant" rather than asserting a connector, an AWS account ID, or a price as permanent.

What you get

• A plain-language Solution Overview of the two AWS sources — Amazon S3 (cross-account IAM role + external ID, read-only + KMS decrypt) and Amazon RDS (Key Vault credential over a network path) — how they scan and classify, and how they complement Microsoft 365 Purview.

• A Requirements Guide with verify / implement-if-missing for every prerequisite: the Purview account, connector availability, least-privilege collection roles, the AWS inventory with owners, the S3 trust path, the RDS Key Vault credential, the integration-runtime network path, a classification approach, and the pilot scope.

• An Implementation Guide of precise, ordered steps: design the collection/role model, establish the S3 trust and RDS credential safely, register the AWS sources, **pilot-scan** a bounded set, review classifications, assign ownership, validate against the gate, then expand with scheduled scans.

• Five PowerShell tools for readiness/role discovery, pilot-scope capture, AWS source registration and scan configuration (`-WhatIf` by default), a scan-and-classification coverage report, and a consolidated readiness-gate check — authentic Purview data-plane REST automation with **no credentials in code** (S3 role ARN and RDS secret referenced by name only).

• Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

AWS data governed from Purview: S3 buckets and RDS databases registered into the Data Map, scanned and classified, and owned — validated on a pilot before you scale.

The deployment model

Assess → Design → Configure → **Pilot (bounded scan)** → Validate → Expand — with a pilot gate before any AWS-wide scanning.

Outcome

The AWS blind spot closed — S3 and RDS discoverable, classified, owned, and governed in the same Purview Data Map as the rest of the estate — with regulatory, records, and access determinations left to the owners who make them.

1. Solution Overview — AWS Data-Source Governance (S3/RDS): Microsoft Purview

What this solution is

Microsoft Purview has two related but distinct sides. The **compliance** side (labels, DLP, retention, insider risk) governs Microsoft 365 content. This solution implements a focused slice of the **data governance** side: **registering, scanning, and classifying your AWS data — Amazon S3 buckets and Amazon RDS databases — into the Purview Data Map**, so AWS data stops being a governance blind spot and becomes a mapped, classified, cataloged part of your estate. It does not re-implement the whole Data Map / Unified Catalog build; it delivers the **AWS-source connector workflow** end-to-end: register the source, prove the trust/credential path, run a bounded pilot scan, review classifications, then expand.

> ***Boundary.*** *This solution provides the technical method to *register, scan, classify, and catalog* AWS S3 and RDS sources. It does **not** make the business or regulatory determinations — which data is "confidential", what the retention or records schedule is, or who may access it — those belong to the organization's data owners, stewards, compliance, privacy, and legal functions. The solution gives them governed, discoverable AWS data to make those decisions against.*

Why this matters

Most organizations govern Microsoft 365 well and their AWS estate not at all. A copy of a customer table lands in an S3 bucket, an application database runs on Amazon RDS, an analyst exports extracts to another bucket — and none of it appears in the data map. Nobody can answer "is there PII in our S3 buckets?", "which RDS databases hold regulated data?", or "who owns this bucket?" — because AWS data was never registered, scanned, or classified. Ungoverned data is un-securable data: you cannot protect, retain, or grant access to something you cannot see. This solution brings AWS S3 and RDS into the same Purview Data Map that already governs the rest of the estate.

The two AWS sources this covers

This solution stands up the AWS connector workflow for two source types. Availability of specific multicloud connectors and their exact configuration is **evolving** in Microsoft Purview — every capability below is flagged "verify your licensing/tenant".

1. **Amazon S3** — object storage buckets. Purview reaches S3 by assuming a **cross-account AWS IAM

role **you create: the role trusts Microsoft's Purview AWS account and is conditioned on an** external ID**, and it grants** read-only **access to the buckets in scope (plus** KMS decrypt **for encrypted buckets). Scanning enumerates objects and applies** classifications** (system classifications such as credit-card or national-ID number, plus any custom classification you define).

1. **Amazon RDS** — managed relational databases (the RDS engines Purview supports — **verify the current supported engine list for your tenant**, historically SQL Server and PostgreSQL). Purview connects with a** database credential referenced from Azure Key Vault by name** over a network path to the instance, enumerates databases/schemas/tables, and applies classifications the same way.

The Data Map, collections, and roles

AWS sources register into the **Data Map** under a **collection** hierarchy that also carries access control. Each collection carries its own role assignments: **Collection admins** (manage the collection and its roles), **Data source admins** (register sources and run scans), **Data curators** (edit catalog metadata and classifications), and **Data readers** (browse the catalog). Least-privilege role assignment is a first-class step, not an afterthought — the Implementation Guide names each role at each stage.

Credentials and network — safe by design

Scanning reaches into live AWS systems, so it needs a trust path and network line of sight — never a secret in a script. This solution follows the safe-by-design pattern Microsoft documents:

- **Amazon S3** uses a **role-based trust** (an AWS IAM role ARN + external ID) rather than a long-lived key; Purview assumes the role. The role ARN is referenced as a Purview credential **by name**; **no AWS secret is stored in a script or document**.
- **Amazon RDS** uses a **database credential stored in an Azure Key Vault** linked to the Purview account and referenced **by name** — again, no secret in code.
- Cloud-reachable sources are scanned through the **Azure (managed) integration runtime**; RDS instances in a private VPC require a **self-hosted integration runtime (SHIR)** (or managed VNet runtime — verify) with network line of sight. Scans are always **piloted on a bounded scope first**.

Licensing and the evolving portal

Microsoft Purview Data Governance (the Data Map and its multicloud connectors) is billed through a **consumption / pay-as-you-go** model tied to an **Azure subscription**, and the capability set is evolving — the **Amazon S3** and **Amazon RDS** connectors, the supported RDS engines, and the exact trust/credential configuration may be in preview or may have changed name, path, or availability. **Verify current pricing, availability, supported engines, and portal paths for your tenant** in Microsoft's documentation and the Purview portal before you plan. This solution cites the method; it never asserts a price, an AWS account ID, or a portal path as permanent.

What this is not

This is the *AWS data-source governance* solution. It is **not** the full Data Map / Unified Catalog build, and it is **not** the Microsoft 365 compliance configuration (sensitivity/retention labels, DLP, insider risk) — those

are separate Purview solutions in this category. It complements them by pulling AWS S3 and RDS into the same governed map.

The deployment model

> *Assess AWS readiness (account, roles, connector availability, network), design the collection and role model and the trust/credential path, configure the AWS source registration,* ***pilot scanning on a bounded scope*** *(a few buckets / one RDS database), validate the results against the gate, then expand across the AWS estate — moving scans from one-time to scheduled only after the pilot passes. The repeatable way:* ***Assess*** *→* ***Design*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand.***

2. Requirements Guide — AWS Data-Source Governance (S3/RDS): Microsoft Purview

Verify every requirement before you register an AWS source or run a scan. Each entry gives Description, Why, How to verify, and How to implement if missing. Category-wide prerequisites (tenant, roles, toolchain, scope, stakeholders, pilot) are in `global-requirements-purview.md` — this guide adds the prerequisites specific to **governing AWS S3 and RDS in the Purview Data Map**.

> ***Licensing/availability caveat.*** *Microsoft Purview Data Governance is billed by a consumption / pay-as-you-go model on an Azure subscription and is evolving; the* ***Amazon S3*** *and* ***Amazon RDS*** *multicloud connectors, the supported RDS engines, and the exact trust/credential configuration may be in preview or may have changed. Where this guide names a capability, an AWS account ID, or a portal path,* ***verify its current availability for your tenant*** *in Microsoft's documentation and the Purview portal before relying on it.*

2.1 A Microsoft Purview account (Data Map) provisioned on an Azure subscription

Description: A Microsoft Purview account — the Data Map resource — provisioned in an **Azure subscription** (distinct from the M365 compliance portal). The account exposes the collection hierarchy, scanning, and the multicloud connectors AWS sources register into. Why: The Data Map and scans live in this account; without it there is nothing to register AWS sources into, and the multicloud connectors are surfaced by this account. How to verify: The account is visible in the Azure portal and its data-plane endpoint (`https://<account>.purview.azure.com`) resolves; run `DGAWS-Discovery.ps1` to confirm reachability. How to implement if missing: Provision a Microsoft Purview account under change control on the target subscription; confirm the consumption billing model and region with your Azure owner.

2.2 Amazon S3 / Amazon RDS connector availability confirmed for your tenant

Description: Confirmation that the **Amazon S3** and/or **Amazon RDS** source connectors are available in your Purview account, and the current supported **RDS engine list** (historically SQL Server and PostgreSQL — verify). Why: Multicloud connector availability and the supported engines are evolving/preview; you cannot register a source type the connector doesn't currently offer in your tenant. How to verify: In the Purview portal, **Data Map** → **Sources** → **Register**, confirm **Amazon S3** and **Amazon**

RDS appear as registerable source types; check Microsoft's current multicloud-connector documentation for the supported RDS engines. How to implement if missing: If a connector isn't offered, verify licensing/region and Microsoft's current availability; scope the pilot to the source type(s) actually available and defer the other.

2.3 Least-privilege data-governance roles on the Data Map collection

Description: Role assignments at the right scope — **Collection admin**, **Data source admin**, **Data curator**, **Data reader** — on the collection the AWS sources will register into. Azure **Owner/Contributor** on the account only where account-level changes are required. Why: Purview uses fine-grained, collection-scoped roles; registering a source and running a scan requires **Data source admin** on that collection, and least-privilege is a governance requirement in its own right. How to verify: In the Purview portal, open the collection's **Role assignments**; run `DGAWS-Discovery.ps1` (reports collections and role coverage, read-only). How to implement if missing: Assign the least-privilege role at the narrowest collection scope under change control; reserve account-level Collection admin for a small named group.

2.4 An AWS account inventory of the S3 buckets and RDS instances in scope, with owners

Description: A written list of the **AWS resources** to govern — the **S3 buckets** (with region and whether they are KMS-encrypted) and the **RDS instances/databases** (engine, endpoint, region) — each with a named **data owner** and **data steward**, and the AWS account ID(s) they live in. Why: You register, scan, and classify per source; classification and ownership are meaningless without a known owner per bucket/database, and the region/encryption drive the IAM and KMS permissions the scan needs. How to verify: For each in-scope bucket/database you can name the resource, its region, its encryption state, and its owner; run `DGAWS-Set-Scope.ps1` to record the pilot source set. How to implement if missing: Inventory the AWS estate with the AWS account owners, and assign a data owner and steward to every bucket/database before registering it.

2.5 Amazon S3 trust path — a cross-account IAM role (ARN + external ID), read-only

Description: For S3, an **AWS IAM role** you create that **trusts Microsoft's Purview AWS account** (the account ID/principal shown in the Purview S3 registration UI — **read it from the portal, do not assume it**) with a **condition on the external ID** Purview provides, and grants **read-only** access to the in-scope buckets (for example the AWS-managed `AmazonS3ReadOnlyAccess`, or a bucket-scoped read policy) plus **KMS decrypt** on the key for any encrypted bucket. The role's **ARN** is referenced as a Purview credential **by name** — no AWS access key is stored. Why: Purview scans S3 by **assuming this role**; the external-ID condition prevents the confused-deputy problem, and read-only + KMS-decrypt is the least privilege a scan needs. A role-based trust avoids long-lived keys entirely. How to verify: The IAM role exists with the trust policy (Microsoft's Purview AWS principal + your external ID) and the read/KMS permissions; the source's **Test connection** in Purview succeeds; the role ARN is registered as a Purview credential by name. How to implement if missing: In the Purview S3 registration UI, copy the **Microsoft AWS account / principal and the external ID**; create the IAM role in AWS with that trust and read-only + KMS permissions; register its ARN as the credential; run **Test connection**.

2.6 Amazon RDS credential via Azure Key Vault — never in code

Description: For RDS, a **database login with read access** to the in-scope databases, whose secret is stored in an **Azure Key Vault** linked to the Purview account and referenced **by name**. **No credential is ever placed in a script or document.** Why: RDS scans authenticate to the database with SQL/DB auth; storing the secret in Key Vault and referencing it by name is mandatory and is how the provided scripts stay secret-free. How to verify: A Key Vault is linked to the Purview account and holds the RDS secret; the RDS source's **Test connection** succeeds using the named credential. How to implement if missing: Link a Key Vault to the account, create a least-privilege read login on the RDS database, store its secret in Key Vault, and verify **Test connection** before scanning.

2.7 Network path to each source (integration runtime + AWS network/firewall)

Description: A working network path from the scanner to each source: the **Azure (managed) integration runtime** for publicly reachable S3 and RDS, or a **self-hosted integration runtime (SHIR)** (or managed VNet runtime — verify) for RDS instances in a **private VPC**; and AWS-side network rules (security groups, RDS publicly-accessible setting, S3 bucket policy/VPC endpoint) that permit the scan. Why: A scan cannot classify what it cannot reach; RDS in a private subnet or an S3 bucket restricted by policy will fail **Test connection** until the network path and AWS rules allow the scanner. How to verify: The integration runtime shows **Running/Available** in the portal and **Test connection** on each source succeeds; `DGAWS-Discovery.ps1` reports registered runtimes. How to implement if missing: Use the managed runtime for reachable sources; install and register a SHIR with line of sight to private RDS, adjust the AWS security group / bucket policy to permit the scan, then re-test.

2.8 A classification approach (system + any custom classifications)

Description: A decision on how AWS assets will be classified: rely on the **built-in system classifications** (credit card, national ID, and the large built-in set) and define any **custom classification rules** the organization needs (for example an internal customer-ID pattern) via the scan rule set. Why: Classification is what turns the raw AWS inventory into a sensitivity-aware map that drives ownership, access, and downstream protection. How to verify: System classifications appear on a test scan result; any custom classification rule is defined and matches sample data. How to implement if missing: Start with system classifications for the pilot; define custom rules only where a real internal pattern isn't covered, and validate each against sample data before estate-wide use.

2.9 A pilot scope and change control

Description: A **bounded pilot** — one collection and one or two AWS sources (a couple of S3 buckets and/or one RDS database) — to prove registration, trust/credential, scanning, and classification before AWS-wide rollout, plus a change-control window (scans consume resources and touch live AWS systems and can incur AWS request/egress cost). Why: Every solution in this category follows Assess → Design → Configure → **Pilot** → Validate → Expand; scanning is resource-, cost-, and access-sensitive and is always proven on a bounded scope first. How to verify: A named pilot collection, its AWS sources, and a change window exist; run `DGAWS-Set-Scope.ps1` to materialize the pilot scope. How to implement if missing: Choose one well-understood, high-value bucket set and/or one RDS database as the pilot; agree the scan window and expected AWS cost with the source owners.

Checklist

- Purview account (Data Map) provisioned on an Azure subscription — `DGAWS-Discovery.ps1`
- Amazon S3 / RDS connector availability + supported RDS engines confirmed for the tenant
- Least-privilege collection roles assigned (Data source admin to register/scan) — `DGAWS-Discovery.ps1`
- AWS buckets/RDS instances inventoried with owners, region, encryption — `DGAWS-Set-Scope.ps1`
- S3 cross-account IAM role (ARN + external ID, read-only + KMS decrypt) created; ARN registered by name
- RDS credential stored in Azure Key Vault, referenced by name (no secrets in code)
- Network path per source (managed IR or SHIR) + AWS security group/bucket policy — Test connection passes
- Classification approach agreed (system + any custom)
- Pilot scope and scan change window (with expected AWS cost) agreed — `DGAWS-Set-Scope.ps1`

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is visible in the Purview portal.
- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names (e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.
- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.
- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.
- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.
- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal** stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements, not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — AWS Data-Source Governance (S3/RDS): Microsoft Purview

Follow these steps in order. This guide assesses AWS readiness, designs the collection and role model, establishes the S3 trust and RDS credential paths safely, registers and **pilot-scans** a bounded AWS source set, reviews classifications, then expands across the AWS estate. Artifact names are fixed — create them with the exact names shown and reuse them everywhere. **Every scan is piloted on a bounded scope before the AWS estate is scanned.**

> *Model: prove the AWS connector, credential/trust path, and classification on a pilot scope, then expand. The repeatable way: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.** Verify current portal paths, connector availability, supported RDS engines, and the Microsoft AWS principal/external ID for your tenant as you go — the multicloud connectors are evolving.*

Step 1 — Assess current state (read-only)

1. Run `DGAWS-Discovery.ps1` to capture readiness signals: the Purview account endpoint is reachable, which **collections** exist and their **role coverage**, which **integration runtimes** are registered, and how many AWS (and other) sources are already registered. No changes are made.

1. In the Purview portal (**Data Map** → **Sources** → **Register**) confirm **Amazon S3** and **Amazon RDS** are offered and note the **supported RDS engines** for your tenant.

1. Record gaps against the Requirements checklist. Do **not** proceed past a missing account, a missing connector, missing least-privilege roles, or a missing network path — remediate those first.

Step 2 — Design the collection and role model

1. Decide the **collection** the AWS sources register into. For the pilot, create one collection named `DGAWS-Collection-Pilot` under the root; plan the broader hierarchy (for example an AWS branch by AWS account or by sensitivity) on paper for the expansion phase.

1. Assign least-privilege roles at the narrowest scope: **Data source admin** on `DGAWS-Collection-Pilot` to whoever will register/scan AWS sources, **Data curator** to the catalog editors, **Data reader** to browsers, and **Collection admin** to a small named group. Reserve account-level admin for a minimal named group.

Step 3 — Establish the S3 trust path (cross-account IAM role, safe by design)

1. In the Purview portal, begin **Register** → **Amazon S3** and read the **Microsoft AWS account / principal and the external ID **it displays.** Copy them from the portal — do not assume an account ID.**

1. In AWS IAM, create a role named (for example) `PurviewScanRole-S3` with a **trust policy** that allows that Microsoft Purview AWS principal to assume it, **conditioned on the external ID**.

1. Grant the role **read-only** access to the in-scope buckets (the AWS-managed `AmazonS3ReadOnlyAccess`, or a tighter bucket-scoped read policy) and `kms:Decrypt` on the KMS key for any encrypted bucket. **Put no AWS secret anywhere** — the trust is role-based.

1. In Purview, register the role's **ARN** as a credential **by name** (for example `DGAWS-Cred-S3-Role`). This name is what the scripts reference; the ARN is never inlined in a script.

Step 4 — Establish the RDS credential path (Key Vault, safe by design)

1. Link an **Azure Key Vault** to the Purview account (or confirm one is linked).

2. Create a **least-privilege read login** on the in-scope RDS database, store its secret in the Key Vault, and create a Purview credential **by name** (for example `DGAWS-Cred-RDS-KV`) that references the Key Vault secret. **No credential is ever placed in a script or document.**

1. Confirm the **integration runtime** for each source: the **Azure (managed) runtime** for reachable S3/RDS, or install and register a **self-hosted integration runtime (SHIR)** for RDS in a private VPC. Confirm it shows **Running/Available**, and adjust the AWS **security group / bucket policy** to permit the scanner.

Step 5 — Record the pilot scope

1. Run `DGAWS-Set-Scope.ps1` to record the pilot AWS source set (a couple of S3 buckets and/or one RDS

database) with owners, plus the pilot collection and scan-rule-set intent (read-only; produces the scope manifest you register against).

Step 6 — Register the pilot AWS sources into the Data Map

1. Register the pilot sources into `DGAWS-Collection-Pilot`. You may use the portal

(**Register** → **Amazon S3 / Amazon RDS**), or run `DGAWS-Register-AwsSource.ps1` which **defaults to** `-WhatIf` and prints exactly what it would register (kind `AmazonS3` / `AmazonRDS`, the bucket or endpoint, the collection, and the credential name); re-run with `-Commit` to register.

1. Run **Test connection** for each registered source and confirm it passes (S3 role assume succeeds;

RDS auth + network path succeed) before scanning. Do not scan until Test connection passes.

Step 7 — Configure and pilot the scan (bounded scope, pilot-first)

1. Define the scan configuration: a **scan rule set** named `DGAWS-ScanRuleSet-Pilot` (start from the

system default rule set), the **classifications** to apply (system classifications plus any custom rule named `DGAWS-Classification-Custom-<name>`), and a **scoped** scan that targets only the pilot buckets/database — not every bucket in the account or every database on the instance.

1. Trigger the scan **once** on the bounded scope (not scheduled yet) via

`DGAWS-Register-AwsSource.ps1 -ConfigureScan` (`-WhatIf` by default; `-Commit` to apply) or the portal. Pass/fail: the scan completes with **status Succeeded** and returns assets with classifications.

1. **Rollback:** if a scan misbehaves, over-reaches, or runs up AWS request/egress cost, cancel the run

in the portal and set the scan to disabled; a scan does not modify source data, but cancel to stop resource/cost consumption. Re-scope and re-run on a smaller set.

Step 8 — Review scanned assets and classifications

1. Run `DGAWS-Report.ps1` to list the pilot AWS assets discovered (S3 objects/paths, RDS

tables/columns), the classifications applied, and any assets returned **unclassified**.

1. With the data owner, confirm the classifications are correct. Where a real internal pattern was

missed, define a **custom classification rule** (`DGAWS-Classification-Custom-<name>`), validate it against sample data, add it to `DGAWS-ScanRuleSet-Pilot`, and re-scan the pilot scope.

Step 9 — Assign ownership on the AWS assets

1. In the catalog, assign the **data owner** and **steward** recorded in the scope manifest to the

scanned buckets/databases so every governed AWS asset has a named owner — ownership is what makes classification and later access decisions accountable.

Step 10 — Validate against the pilot gate

1. Complete the Testing Guide. Run `DGAWS-Test-Readiness.ps1` for a consolidated PASS/GAP gate check.

The pilot passes only when: the AWS sources are registered and **Test connection** passes; the scan **Succeeded** and returned classified assets; and every scanned pilot asset has an owner assigned.

1. Remediate any gaps and re-run the relevant validation before proceeding. Do **not** widen scope until the gate passes.

Step 11 — Expand across the AWS estate (enforce)

1. After the pilot passes, register further S3 buckets and RDS databases one grouping at a time (for example per AWS account), re-running `DGAWS-Set-Scope.ps1`, `DGAWS-Register-AwsSource.ps1`, and `DGAWS-Report.ps1` for each new scope. Reuse the same IAM role pattern (extending its bucket/KMS scope) and Key Vault credential pattern.

1. Move scans from one-time to **scheduled** only once a scope is validated, so classifications stay current as AWS data changes — and watch AWS request/egress cost as scope grows.

1. Keep ownership assigned on every newly scanned asset and track coverage with `DGAWS-Report.ps1`.

Order of operations

```
Assess -> Design (collection/roles) -> S3 trust (IAM role + external ID) -> RDS credential (Key Vault) & network -> Record scope -> Register AWS sources -> Pilot scan (bounded) -> Review classifications -> Assign ownership -> Validate gate -> Expand (scheduled scans)
```

5. Testing Guide — AWS Data-Source Governance (S3/RDS): Microsoft Purview

Run these tests after the pilot scan (Implementation Steps 6–9) and again after each expansion scope. Each test confirms one part of the AWS-governance workflow works end to end. All read-only checks are safe to repeat.

Test 1 — Account and roles are ready

Do: Run `DGAWS-Discovery.ps1`. Expected: The Purview account endpoint is reachable, `DGAWS-Collection-Pilot` exists, and least-privilege roles (Collection admin, Data source admin, Data curator, Data reader) are reported as assigned; **Data source admin** (needed to register/scan) is present. No required role is missing.

Test 2 — Amazon S3 / RDS connectors are available

Do: In the Purview portal, open **Data Map** → **Sources** → **Register**. Expected: **Amazon S3** and **Amazon RDS** appear as registerable source types for your tenant, and the supported RDS engine list matches what

the pilot database uses.

Test 3 — The S3 trust path works (role assume + Test connection)

Do: On the registered S3 source, run **Test connection**; confirm the credential references the role ARN by name. Expected: Purview successfully **assumes the cross-account IAM role** (trust + external ID correct) and **Test connection passes**; encrypted buckets resolve (KMS decrypt granted). No AWS secret is stored anywhere.

Test 4 — The RDS credential path works (Key Vault + network + Test connection)

Do: On the registered RDS source, run **Test connection**; confirm the credential references the Key Vault secret by name and the integration runtime shows Running/Available. Expected: Purview authenticates to the RDS database using the **named Key Vault credential** over a working network path (managed IR or SHIR) and **Test connection passes**.

Test 5 — The scan succeeds and returns assets

Do: Review the pilot scan run in the portal and run `DGAWS-Report.ps1`. Expected: The scan status is **Succeeded**, the run scanned the intended bounded scope (the pilot buckets/database, not the whole account/instance), and assets (S3 paths/objects, RDS tables) were returned into the Data Map.

Test 6 — AWS assets are classified

Do: Run `DGAWS-Report.ps1` and review the classification coverage for the pilot AWS assets. Expected: System (and any custom) classifications are applied to the assets that contain matching data; the count of unexpectedly **unclassified** assets is understood and accepted by the data owner.

Test 7 — Scanned AWS assets have owners

Do: Open the pilot buckets/databases in the catalog and run `DGAWS-Report.ps1`. Expected: Every scanned pilot asset has a named **owner/steward** assigned (from the scope manifest); no governed AWS asset is unowned.

Test 8 — Consolidated readiness gate

Do: Run `DGAWS-Test-Readiness.ps1 -PurviewAccount <account> -Collection DGAWS-Collection-Pilot`. Expected: A single PASS/GAP summary confirming account reachable, roles present, AWS sources registered and Test-connection-ready, at least one scan Succeeded, classification coverage > 0, and owners assigned — the pilot gate in one view.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–7 pass: connectors available, S3 role-assume and RDS Key Vault credential Test connections pass, scan Succeeded with classified assets, and every scanned asset has an owner (Test 8 shows all PASS)
Expand pass	Tests 1–7 pass for each new AWS scope, and scheduled scans are enabled only after the scope validates (with expected AWS request/egress cost accepted)

6. Troubleshooting Guide — AWS Data-Source Governance (S3/RDS): Microsoft Purview

Use `DGAWS-Discovery.ps1` for readiness/role/runtime and AWS-source state, `DGAWS-Report.ps1` for scan status and classification/ownership coverage, `DGAWS-Set-Scope.ps1` to (re)record a scope, `DGAWS-Register-AwsSource.ps1` (`-WhatIf` by default) to register/scan, and `DGAWS-Test-Readiness.ps1` for the consolidated pilot gate. All scripts are read-only unless you pass `-Commit` (only `DGAWS-Register-AwsSource.ps1` writes).

A. Cannot reach the account / token fails

Symptom	Cause	Remediation
`Discovery` reports endpoint GAP	Wrong account name or endpoint	Confirm the account name; the endpoint is `https://<account>.purview.azure.com`; re-run `DGAWS-Discovery.ps1`
Token acquisition fails	Not signed in / wrong resource	Run `Connect-AzAccount`; the scripts request a token for `https://purview.azure.net` — confirm your account can obtain it
401/403 on every call	Missing least-privilege role	Assign Data reader (browse) or the specific role the action needs at the collection scope; re-run
Calls fail with "api-version" errors	The example API version has moved	The Data Governance experience is evolving — verify and set the current data-plane API version for your tenant

B. Amazon S3 / Amazon RDS connector isn't offered

Symptom	Cause	Remediation
No **Amazon S3** or **Amazon RDS** in Register	Connector unavailable/preview for your tenant	Verify licensing/region and Microsoft's current multicloud-connector availability; scope the pilot to the connector(s) actually offered

Symptom	Cause	Remediation
RDS engine not registerable	Engine not in the supported list	Check Microsoft's current supported RDS engine list; if unsupported, defer that database or use an alternative in-scope engine
Unknown source "kind"	Connector name differs from `AmazonS3`/`AmazonRDS`	Verify the exact `kind`/`credentialType` names for your tenant before `-Commit`

C. S3 source registration or Test connection fails

Symptom	Cause	Remediation
Register returns 403	Not a Data source admin on the collection	Assign Data source admin on `DGAWS-Collection-Pilot`; re-run `DGAWS-Register-AwsSource.ps1 -Commit`
Test connection fails: "assume role" denied	Trust policy wrong (Microsoft principal or external ID)	Re-copy the **Microsoft AWS principal and external ID** from the Purview S3 registration UI; fix the IAM role trust policy; re-test
Test connection fails: access denied on bucket	Role lacks S3 read	Grant `AmazonS3ReadOnlyAccess` or a bucket-scoped read policy to the role; re-test
Encrypted bucket returns nothing / KMS error	Role lacks `kms:Decrypt`	Grant `kms:Decrypt` on the bucket's KMS key to the role; re-test
"Credential not found"	Credential name wrong / role ARN not registered	Register the role ARN as a Purview credential **by name** and pass that name (`-Credential`); never inline an ARN or key

D. RDS source registration or Test connection fails

Symptom	Cause	Remediation
Test connection fails: auth	Wrong/expired DB login or Key Vault secret	Confirm the RDS read login and the Key Vault secret; reference the credential **by name** (`-Credential`); re-test
Test connection fails: network/timeout	RDS in a private VPC, no path	Install and register a self-hosted integration runtime (SHIR) with line of sight; open the RDS security group to the scanner; re-test

Symptom	Cause	Remediation
"Credential not found" / Key Vault error	Vault not linked or secret name wrong	Link the Key Vault to the account, store the secret, reference it by name; never inline a secret
Publicly-inaccessible instance	RDS `publicly accessible` off + managed IR used	Use a SHIR/managed VNet runtime with VPC access, or adjust network per your security policy

E. Scan runs but classifies nothing

Symptom	Cause	Remediation
Assets returned, 0 classified	Scan rule set has no classifications	Confirm `DGAWS-ScanRuleSet-Pilot` includes system classifications; re-scan and review with `DGAWS-Report.ps1`
An internal pattern isn't detected	No custom classification rule	Define `DGAWS-Classification-Custom-<name>`, validate against sample data, add to the rule set, re-scan
Scan scope too broad / slow / costly	Whole bucket set or instance scanned	Re-scope the scan to the bounded pilot buckets/database; keep it one-time until validated
Scan status Failed	Transient/auth/network	Open the run in the portal for the failure reason; fix credential/runtime/AWS network; re-run

F. AWS request/egress cost higher than expected

- S3 scanning issues list/get requests and can incur AWS request and egress charges; RDS scanning

consumes instance resources. Keep scans **one-time and bounded** until validated, scope tightly, and agree the expected cost with the AWS account owner as part of change control. Cancel a runaway scan in the portal (no source data is modified) and re-scope.

G. Classifications look wrong

- False positives/negatives are expected on first scan. Review with the **data owner** using

`DGAWS-Report.ps1`; tune or remove a custom classification rule and re-scan the pilot scope. The solution provides the technical result — the owner decides whether a classification is correct for the business.

H. Ownership / gate not passing

Symptom	Cause	Remediation
`DGAWS-Test-Readiness.ps1` shows "not owned"	Owners not assigned on scanned assets	Assign the owner/steward from the scope manifest to each bucket/database in the catalog; re-run
Gate shows no Succeeded scan	Scan not run or Failed	Trigger the one-time scan; fix any Test-connection failure first; re-check
Gate shows classification 0	Rule set / scope issue	See section E; re-scan and re-check

I. Diagnostics quick reference

```
.\DGAWS-Discovery.ps1 -PurviewAccount contoso-purview                # readiness: endpoint, collections,
.\DGAWS-Set-Scope.ps1 -S3Bucket @('contoso-fin-extracts') -RdsDatabase @('fin-rds-01.contoso.example/Finan
.\DGAWS-Register-AwsSource.ps1 -PurviewAccount contoso-purview -Collection DGAWS-Collection-Pilot `
    -SourceName s3-fin-extracts -Kind AmazonS3 -BucketOrEndpoint contoso-fin-extracts -Credential DGAWS-Cr
.\DGAWS-Report.ps1 -PurviewAccount contoso-purview -Collection DGAWS-Collection-Pilot   # scan + classific
.\DGAWS-Test-Readiness.ps1 -PurviewAccount contoso-purview -Collection DGAWS-Collection-Pilot  # consolida
```

J. When to escalate

If the account is reachable, roles are assigned, the runtime is Running, the S3 role-assume/RDS credential Test connection passes, and a scoped scan still returns no classified assets after tuning the rule set, capture the `DGAWS-Discovery.ps1` and `DGAWS-Report.ps1` output and the portal scan-run detail, and review the source connectivity and rule set with the data owner and your Purview/Azure administrator (and your AWS account owner for IAM/KMS/network). For evolving or preview connectors, verify current availability, supported engines, and API versions against Microsoft's documentation before assuming a defect.

7. Best Practices — AWS Data-Source Governance (S3/RDS): Microsoft Purview

- Confirm connector availability before you design anything. The Amazon S3 and Amazon RDS connectors,

the supported RDS engines, and the Microsoft AWS principal/external ID are evolving — read them from the Purview registration UI and Microsoft's current docs for your tenant rather than assuming a value that may have moved.

- Prefer role-based trust over keys for S3. A cross-account IAM role (ARN + external ID, assumed by

Purview) avoids long-lived AWS access keys entirely; the external-ID condition prevents the confused-deputy problem. Never store an AWS access key in a script, a scan, or a document.

- Scope the S3 IAM role to read-only and the buckets in scope. Grant `AmazonS3ReadOnlyAccess` only if

you must; a bucket-scoped read policy plus `kms:Decrypt` on just the encrypted buckets in scope is the least privilege a scan needs. Widen the role deliberately as you expand.

- Keep RDS secrets in Key Vault, referenced by name. Link an Azure Key Vault to the account, store the

RDS read login there, and reference it by name — this is both the secure pattern and what keeps the provided scripts secret-free. Use a least-privilege read login, not an admin account.

- Pilot every scan on a bounded scope. Scoped, one-time scans on a couple of buckets or one database

prove classification, connectivity, and **AWS request/egress cost** before you point a scan at an entire account or instance; move to scheduled scans only after a scope validates.

- Mind AWS cost and blast radius. S3 scanning issues list/get requests and can incur request and egress

charges; scope scans, keep them one-time until validated, and agree the expected cost with the AWS account owner as part of change control.

- Trust system classifications first, add custom sparingly. The built-in classifications cover most

regulated data; define a custom classification rule (`DGAWS-Classification-Custom-<name>`) only for a real internal pattern, and validate it against sample data before AWS-wide use.

- Every scanned AWS asset gets an owner and a steward. Ownership is what makes classification and access

decisions accountable; an unowned bucket or database in the catalog is a governance gap, not a governed asset.

- Right-size the network path. Use the Azure (managed) integration runtime for reachable S3/RDS; stand

up a self-hosted integration runtime (SHIR) with line of sight for RDS in a private VPC, and open only the security-group/bucket-policy rules the scanner needs — no broader.

- Treat determinations as someone else's job. The solution registers, scans, classifies, and catalogs

AWS data; the business/records/legal owners decide sensitivity, retention, and access. Keep that separation explicit so technical scanning is never mistaken for a compliance ruling.

- Verify licensing, pricing, and portal paths as you go. The multicloud connectors and the consumption

billing are evolving — confirm current availability and pricing in Microsoft's docs and the portal rather than assuming a path, an AWS account ID, or a price is permanent.

8. Deliverables Summary — AWS Data-Source Governance (S3/RDS): Microsoft Purview

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	Registering, scanning, and classifying AWS S3 and RDS into the Purview Data Map — the two connectors, the safe trust/credential model, and how it complements M365 Purview (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: design, S3 trust + RDS credential, register, pilot-scan, classify, assign ownership, validate, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

Named artifacts

Artifact	Name
Pilot collection	`DGAWS-Collection-Pilot`
S3 role credential (references the IAM role ARN by name)	`DGAWS-Cred-S3-Role`
RDS credential (references the Key Vault secret by name)	`DGAWS-Cred-RDS-KV`
Scan rule set	`DGAWS-ScanRuleSet-Pilot`
Custom classification rule (as needed)	`DGAWS-Classification-Custom-<name>`
Example S3 IAM role (created in AWS)	`PurviewScanRole-S3`

PowerShell scripts

Script	What it does
`DGAWS-Discovery.ps1`	Reports readiness: account endpoint reachable, collections + role coverage, registered integration runtimes, and registered AWS (S3/RDS) source counts (Purview data-plane REST via an Azure token; read-only)
`DGAWS-Set-Scope.ps1`	Records the pilot/expansion AWS scope — the S3 buckets / RDS databases, owners, collection, and scan-rule-set intent — as a scope manifest to configure against (read-only)

Script	What it does
`DGAWS-Register-AwsSource.ps1`	Registers an Amazon S3 or Amazon RDS source and configures a scoped scan; **defaults to** `-WhatIf` (prints the plan), applies only with `-Commit`; credentials referenced by name — no secrets in code
`DGAWS-Report.ps1`	Scan + classification coverage report for AWS assets: scan run status per source, classified vs unclassified assets, and owner coverage (read-only)
`DGAWS-Test-Readiness.ps1`	Consolidated PASS/GAP pilot-gate check across account, roles, AWS source registration/Test-connection readiness, scan status, classification coverage, and ownership (read-only)

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable method following the model: Assess → Design → Configure → Pilot → Validate → Expand.

Outcome

AWS data governed from Purview — S3 buckets and RDS databases registered into the Data Map, scanned and classified, and owned — proven on a bounded pilot before you scale, with the AWS blind spot closed and regulatory, records, and access determinations left to the owners who make them.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DGAWS-Discovery.ps1

Purpose: Reports readiness so you can assess current state before registering AWS sources or scanning: whether the Microsoft Purview account data-plane endpoint is reachable, which collections exist and their role-assignment coverage (especially Data source admin, needed to register/scan), which integration runtimes are registered, and how many Amazon S3 / Amazon RDS sources are already registered. Read-only — it makes no change to the account, sources, or catalog. Supports the Assess step. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGAWS-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az PowerShell modules (Az.Accounts). Sign in first with Connect-AzAccount using an account holding at least Data reader on the collections. The script acquires a data-plane token for the Purview REST API (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS A readiness summary: endpoint reachable (PASS/GAP), collection count and per-collection role coverage, registered integration runtimes, and a count of registered Amazon S3 / Amazon RDS sources — with a PASS/GAP line per signal to record against the Requirements checklist. .NOTES Purview — AWS Data-Source Governance. Read-only against the Purview data-plane REST API. Verify your licensing/tenant: the Data Governance experience, its endpoints, and the AWS connectors are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$PurviewAccount, # short account name, e.g. contoso-purview [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGAWS-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az PowerShell modules (Az.Accounts). Sign in first with Connect-AzAccount using an account holding at least Data reader on the collections. The script acquires a data-plane token for the Purview REST API (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS A readiness summary: endpoint reachable (PASS/GAP), collection count and per-collection role coverage, registered integration runtimes, and a count of registered Amazon S3 / Amazon RDS sources — with a PASS/GAP line per signal to record against the Requirements checklist. .NOTES Purview — AWS Data-Source Governance. Read-only against the Purview data-plane REST API. Verify your licensing/tenant: the Data Governance experience, its endpoints, and the AWS connectors are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$PurviewAccount, # short account name, e.g. contoso-purview [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGAWS-Register-AwsSource.ps1

Purpose: Registers an AWS data source (-Kind AmazonS3 or AmazonRDS) into a Data Map collection and (with -ConfigureScan) prepares a scoped, one-time scan using a named scan rule set. SAFE BY DEFAULT: with no -Commit it runs in -WhatIf mode and only prints the exact registration/scan payload it WOULD send — it changes nothing. Pass -Commit to apply. No AWS secret is ever placed in this script: S3 uses a cross-account IAM role (ARN + external ID) referenced as a Purview credential BY NAME; RDS uses a Key Vault credential referenced BY NAME. HOW TO RUN Preview an S3 registration (default, no change): powershell -ExecutionPolicy Bypass -File .\DGAWS-Register-AwsSource.ps1 -PurviewAccount contoso-purview `-Collection DGAWS-Collection-Pilot -SourceName s3-fin-extracts -Kind AmazonS3` -BucketOrEndpoint contoso-fin-extracts -Credential DGAWS-Cred-S3-Role Preview an RDS registration: ... -SourceName rds-fin-01 -Kind AmazonRDS -BucketOrEndpoint fin-rds-01.contoso.example ` -Database FinanceDB -Credential DGAWS-Cred-RDS-KV Apply + configure

scan: ... same args ... -ConfigureScan -ScanRuleSet DGAWS-ScanRuleSet-Pilot -Commit Requires Az.Accounts and Connect-AzAccount with Data source admin on the collection. EXPECTED RESULTS Default: a printed "[WhatIf] would register / would configure scan" plan and NO change. With -Commit: the source is registered (HTTP 200/201) and, with -ConfigureScan, a one-time scan definition is created; the script prints the resulting object names. Run Test connection in the portal, then trigger/monitor the scan; review with DGAWS-Report.ps1. .NOTES Purview — AWS Data-Source Governance. Writes gated behind -Commit; -WhatIf is the default. Verify the exact source "kind" (AmazonS3 / AmazonRDS), the supported RDS engines, and the current scan API versions for your tenant in Microsoft's docs — the multicloud connectors are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory = $true)][string]$Collection, [Parameter(Mandatory = $true)][string]$SourceName, [Parameter(Mandatory = $true)][ValidateSet('AmazonS3','AmazonRDS')][string]$Kind, [Parameter(Mandatory = $true)][string]$BucketOrEndpoint, # S3 bucket name, or RDS instance endpoint [string]$Database, # RDS only: database name [Parameter(Mandatory = $true)][string]$Credential, # NAME of the Purview credential (S3 role ARN, or RDS Key Vault secret) [switch]$ConfigureScan, [string]$ScanRuleSet = 'DGAWS-ScanRuleSet-Pilot', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

How to run: Preview an S3 registration (default, no change): powershell -ExecutionPolicy Bypass -File .\DGAWS-Register-AwsSource.ps1 -PurviewAccount contoso-purview `-Collection DGAWS-Collection-Pilot -SourceName s3-fin-extracts -Kind AmazonS3` -BucketOrEndpoint contoso-fin-extracts -Credential DGAWS-Cred-S3-Role Preview an RDS registration: ... -SourceName rds-fin-01 -Kind AmazonRDS -BucketOrEndpoint fin-rds-01.contoso.example ` -Database FinanceDB -Credential DGAWS-Cred-RDS-KV Apply + configure scan: ... same args ... -ConfigureScan -ScanRuleSet DGAWS-ScanRuleSet-Pilot -Commit Requires Az.Accounts and Connect-AzAccount with Data source admin on the collection. EXPECTED RESULTS Default: a printed "[WhatIf] would register / would configure scan" plan and NO change. With -Commit: the source is registered (HTTP 200/201) and, with -ConfigureScan, a one-time scan definition is created; the script prints the resulting object names. Run Test connection in the portal, then trigger/monitor the scan; review with DGAWS-Report.ps1. .NOTES Purview — AWS Data-Source Governance. Writes gated behind -Commit; -WhatIf is the default. Verify the exact source "kind" (AmazonS3 / AmazonRDS), the supported RDS engines, and the current scan API versions for your tenant in Microsoft's docs — the multicloud connectors are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory = $true)][string]$Collection, [Parameter(Mandatory = $true)][string]$SourceName, [Parameter(Mandatory = $true)][ValidateSet('AmazonS3','AmazonRDS')][string]$Kind, [Parameter(Mandatory = $true)][string]$BucketOrEndpoint, # S3 bucket name, or RDS instance endpoint [string]$Database, # RDS only: database name [Parameter(Mandatory = $true)][string]$Credential, # NAME of the Purview credential (S3 role ARN, or RDS Key Vault secret) [switch]$ConfigureScan, [string]$ScanRuleSet = 'DGAWS-ScanRuleSet-Pilot', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

DGAWS-Report.ps1

Purpose: Reports the health of the governed AWS estate for a scope: latest scan run status per Amazon S3 / Amazon RDS source, counts of classified vs unclassified AWS assets, and ownership coverage (assets with an owner/steward). Read-only — it queries the Purview data-plane REST / catalog search and changes nothing. Use it to review the pilot (Testing Guide) and to track coverage as the AWS estate expands. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGAWS-Report.ps1 -PurviewAccount

contoso-purview ` -Collection DGAWS-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A coverage summary: per-source last scan status (Succeeded/Failed/Running) for AWS sources, classified vs unclassified asset counts with a classification-coverage %, and ownership coverage — PASS/GAP flags you can record against the pass/fail gate. .NOTES Purview — AWS Data-Source Governance. Read-only. Verify current search/scan API versions for your tenant; the Data Governance experience and its endpoints are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGAWS-Report.ps1 -PurviewAccount contoso-purview ` -Collection DGAWS-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A coverage summary: per-source last scan status (Succeeded/Failed/Running) for AWS sources, classified vs unclassified asset counts with a classification-coverage %, and ownership coverage — PASS/GAP flags you can record against the pass/fail gate. .NOTES Purview — AWS Data-Source Governance. Read-only. Verify current search/scan API versions for your tenant; the Data Governance experience and its endpoints are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGAWS-Set-Scope.ps1

Purpose: Captures and writes a scope manifest for the AWS pilot (or an expansion wave): the target collection, the S3 buckets and/or RDS databases to register (with region/owner/steward and the credential name each uses), and the scan-rule-set intent. Read-only against the tenant — it writes only a local JSON manifest you register and scan against; it does not change the Purview account or any AWS resource, and it holds NO AWS secret (credentials are referenced by name). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGAWS-Set-Scope.ps1 -Collection DGAWS-Collection-Pilot `-S3Bucket @('contoso-fin-extracts','contoso-customer-exports')` -RdsDatabase @('fin-rds-01.contoso.example/FinanceDB') `-S3Credential DGAWS-Cred-S3-Role -RdsCredential DGAWS-Cred-RDS-KV` -OutFile .\aws-pilot-scope.json EXPECTED RESULTS An aws-pilot-scope.json manifest listing the collection, the S3 buckets and RDS databases (each with owner/steward placeholders and the credential name), and the scan rule set, plus a printed summary — used as the input to DGAWS-Register-AwsSource.ps1. .NOTES Purview — AWS Data-Source Governance. Local manifest only; no tenant or AWS change; no secrets. Names are white-label placeholders (contoso.example). Assign a real owner/steward before registering. > [CmdletBinding()] param([string]$Collection = 'DGAWS-Collection-Pilot', [string[]]$S3Bucket = @(), # bucket names, e.g. 'contoso-fin-extracts' [string[]]$RdsDatabase = @(), # 'endpoint/Database', e.g. 'fin-rds-01.contoso.example/FinanceDB' [string]$S3Credential = 'DGAWS-Cred-S3-Role', # NAME of the Purview credential referencing the IAM role ARN [string]$RdsCredential = 'DGAWS-Cred-RDS-KV', # NAME of the Purview credential referencing the Key Vault secret [string]$ScanRuleSet = 'DGAWS-ScanRuleSet-Pilot', [string]$OutFile = '.\aws-pilot-scope.json')

How to run: powershell -ExecutionPolicy Bypass -File .\DGAWS-Set-Scope.ps1 -Collection DGAWS-Collection-Pilot `-S3Bucket @('contoso-fin-extracts','contoso-customer-exports')` -RdsDatabase @('fin-rds-01.contoso.example/FinanceDB') `-S3Credential DGAWS-Cred-S3-Role -RdsCredential DGAWS-Cred-RDS-KV` -OutFile .\aws-pilot-scope.json EXPECTED RESULTS An aws-pilot-scope.json manifest listing the collection, the S3 buckets and RDS databases (each with owner/steward placeholders and the credential name), and the scan rule set, plus a printed summary —

used as the input to DGAWS-Register-AwsSource.ps1. .NOTES Purview — AWS Data-Source Governance. Local manifest only; no tenant or AWS change; no secrets. Names are white-label placeholders (contoso.example). Assign a real owner/steward before registering. > [CmdletBinding()] param([string]$Collection = 'DGAWS-Collection-Pilot', [string[]]$S3Bucket = @(), # bucket names, e.g. 'contoso-fin-extracts' [string[]]$RdsDatabase = @(), # 'endpoint/Database', e.g. 'fin-rds-01.contoso.example/FinanceDB' [string]$S3Credential = 'DGAWS-Cred-S3-Role', # NAME of the Purview credential referencing the IAM role ARN [string]$RdsCredential = 'DGAWS-Cred-RDS-KV', # NAME of the Purview credential referencing the Key Vault secret [string]$ScanRuleSet = 'DGAWS-ScanRuleSet-Pilot', [string]$OutFile = '.\aws-pilot-scope.json')

DGAWS-Test-Readiness.ps1

Purpose: Runs the AWS pilot gate in one view: confirms the Purview account endpoint is reachable, the least-privilege roles (especially Data source admin) are present on the collection, at least one Amazon S3 / Amazon RDS source is registered, each AWS source has at least one scan whose last run Succeeded, classification coverage is > 0, and scanned assets have owners. Read-only — it makes no change. Use it in the Validate step to decide whether the pilot passes before expanding. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGAWS-Test-Readiness.ps1 -PurviewAccount contoso-purview ` -Collection DGAWS-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A single PASS/GAP table across the gate signals and an overall verdict (PILOT PASS only when all gate signals PASS). Any GAP names the requirement to remediate before expanding scope. .NOTES Purview — AWS Data-Source Governance. Read-only gate check; no writes. Verify current Purview data-plane API versions for your tenant — the Data Governance experience and AWS connectors evolve. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection = 'DGAWS-Collection-Pilot', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGAWS-Test-Readiness.ps1 -PurviewAccount contoso-purview ` -Collection DGAWS-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A single PASS/GAP table across the gate signals and an overall verdict (PILOT PASS only when all gate signals PASS). Any GAP names the requirement to remediate before expanding scope. .NOTES Purview — AWS Data-Source Governance. Read-only gate check; no writes. Verify current Purview data-plane API versions for your tenant — the Data Governance experience and AWS connectors evolve. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection = 'DGAWS-Collection-Pilot', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")